AI Resume Screening & Ranking System

How the system creates value for the company, for its users, and for me as its builder

50	39	11	10s	68
resumes processed	eligible & ranked	rejected with reasons	end-to-end run	automated tests

Prepared by **MD Nawaj Saqlain Rizvi**

SDE Intern Coding Assignment submission · October 2026

Figures in this document are generated from *output/results.json* of the run on the provided 50-resume dataset (scoring mode: deterministic (no LLM configured)). Re-run *docs/build_value_doc.py* after a new screening to refresh them.

1. Executive summary

Hiring teams receive far more resumes than they can read carefully, and the first pass is where the most time is spent and the least consistency is achieved. This system automates that first pass for an SDE internship that needs strong Python and real AI/agentic experience. It reads a folder of resumes, removes profiles that do not meet the hard requirements, scores the rest on a transparent 100-point model, enriches them with public GitHub activity, and returns a ranked shortlist in which **every score and every rejection comes with the evidence behind it**.

The guiding design choice is simple: a technology mentioned inside a project or job counts as evidence of use, while the same word in a skills list is only a claim. That one rule is what keeps a well-written but shallow resume from outranking one that shows real work.

1. Ingest	2. Parse	3. Filter	4. Score	5. Enrich	6. Rank
PDF / DOCX / TXT, dual extractor, link annotations	sections, projects, name, email, GitHub user	rules only: Python + applied AI evidence	100-pt model, penalties, evidence lines	GitHub activity, optional LLM judgment	JSON + report, reasons for every decision

On the provided dataset: 50 resumes processed in 10 seconds with 0 failures. 39 candidates ranked, 11 rejected with explicit reasons, 3 candidates penalised for thin or undocumented AI projects, and 35 profiles enriched with live GitHub data.

2. Value to the company

The company's goals are to hire well, hire quickly, and be able to justify its decisions. The table maps each goal to what the system does and where that is visible in this build.

Business outcome	How the system delivers it	Evidence in this build
Faster first-pass screening	Processes a whole folder in one run; reviewers only read the shortlist and the flagged edge cases.	50 resumes in 10s. If a reviewer spends ~3 minutes per resume (an assumption, not a measurement), that is ~2.5 hours of first-pass reading moved to the machine.
Better shortlists	Rewards demonstrated AI systems (agents, retrieval, state, evaluation) over keywords; penalises thin LLM wrappers and tutorial projects; caps classic-ML-only profiles.	3 candidates penalised for wrapper/no-detail projects; 6 classic-ML-only profiles capped at 15/40 AI points.
Consistency and fairness	The same rules apply to every resume in the same order. Writing style, formatting and buzzwords earn nothing on their own.	Eligibility is pure rules (no LLM), covered by unit tests on edge cases such as 'AI-assisted development' and skills-only claims.
Fair even when services fail	If an external service fails for only some candidates, the system does not let that luck decide the ranking.	When GitHub rate limits hit part of the batch, GitHub points are withheld from everyone and a warning is recorded; the fetched data stays visible.
Auditability	Every decision is traceable: matched skills, the project lines that triggered points, applied penalties, and rejection reasons.	results.json stores evidence lines, per-project verdicts, the rule score and LLM score separately, and GitHub status.

Business outcome	How the system delivers it	Evidence in this build
Controlled cost and risk	Runs fully free in deterministic mode. The LLM is optional (paid Claude, the free Gemini tier, or a local model), used only on eligible resumes, cached, and limited to half of one category.	Bounded concurrency (measured 4.8x faster GitHub enrichment and 3.8x faster LLM calls than sequential), disk cache, rate-limit detection, and per-resume fallback when a call fails.
Easy to adopt and change	CLI or REST API; policy is in configuration (weights, thresholds, classic-ML policy, model name), so it can be tuned for other roles without touching logic.	config.py and lexicon.py hold all numbers and term lists; the FastAPI endpoints wrap the same pipeline.

3. Value to the users

The system has two kinds of users: the people who run and read the screening (recruiters, hiring managers, interviewers), and the candidates whose resumes it reads. It is designed to serve both.

Recruiters and hiring managers

Need	What the system gives them
"Why is this person ranked #3?"	A score breakdown across five categories, the strongest AI project with a one-line evidence quote, strengths, and concerns such as "No evidence of Redis" or "Thin LLM wrapper: AI Poem Generator".
"Why was this person rejected?"	Plain-language reasons, e.g. "No evidence of Python stack (profile is JavaScript, React, Next.js)" or "AI terms appear only in skills/summary". This makes a quick human double-check easy.
Interview preparation	Concerns double as interview questions: probe the skills listed but never shown in a project, or ask how a 'deep' project handled evaluation and failure cases.
Trust in automation	Rule and LLM scores are shown side by side; if the LLM disagrees with the rules, the candidate is flagged for manual verification rather than silently moved.
Working with messy input	Corrupt, empty, unsupported and duplicate files are listed separately instead of stopping the run.
Reading the results	A self-contained HTML report: score bars per category, and a click on any name shows the evidence, strengths, concerns and GitHub details. JSON and Markdown versions are produced in the same run.

Candidates

Candidates benefit from being judged on what they have built. A student with a real LangGraph or RAG project and a plain resume is not out-scored by someone with a polished skills section. Mixed stacks are never penalised: JavaScript, Java or React experience does not cause rejection when the Python and AI requirements are met. A missing or private GitHub profile never affects eligibility, and a GitHub API failure is recorded instead of counted against them.

4. Value to me

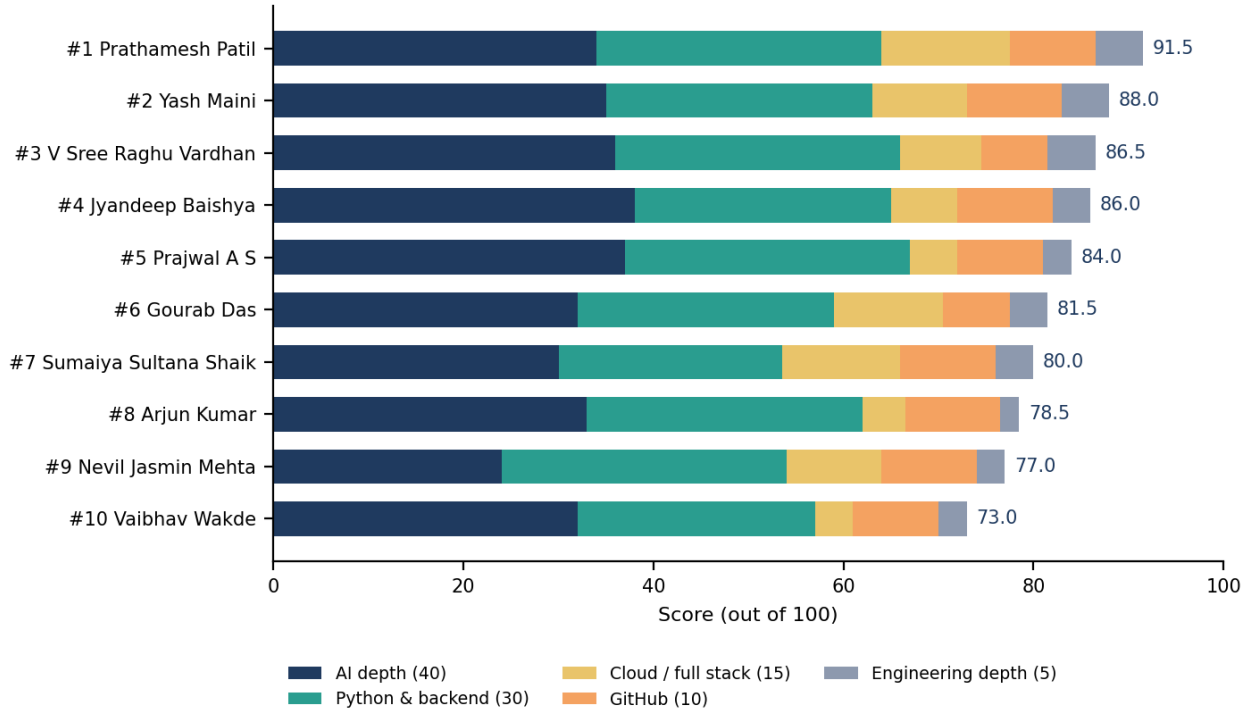
I am a final-year B.Tech CSE student at KIIT, currently an AI/ML Engineer Intern, aiming for roles in GenAI and agentic systems. This project is useful to me in four concrete ways.

What it demonstrates	Where it shows in the code
Understanding of what real AI systems are: the difference between an LLM call and an agentic or retrieval system with state, tools and evaluation.	The AI-depth model scores agentic, retrieval, state, evaluation and data-pipeline signals and penalises thin wrappers; the LLM prompt encodes the same rubric.
Production engineering, not just a demo: failure isolation, bounded concurrency, caching, rate-limit handling, configuration separated from logic.	pipeline.py, github.py and llm.py; per-file and per-call fallbacks; retries with backoff; structured output validated by Pydantic.
Engineering judgment under ambiguity: turning a loose brief into explicit, testable rules, and finding real data problems before they became scoring bugs.	Dual PDF extractors (collapsed words, scrambled order), hyperlink-annotation parsing for GitHub, and 68 tests on the decisions that matter.
Testing that catches real bugs: exercising integrations through the real SDK, not just mocks of my own code.	The LLM adapter test caught that the current Anthropic SDK rejects the *temperature* parameter, a bug that would have silently disabled hybrid scoring.

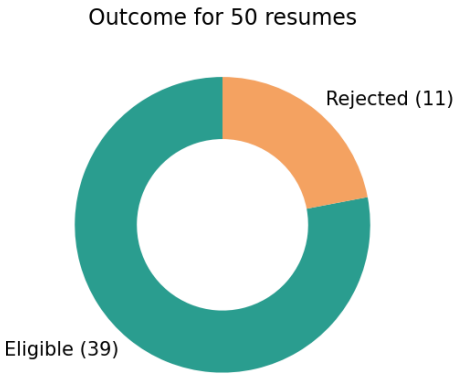
Beyond this assignment, it is a portfolio piece I can walk through in interviews, and a base I can extend (calibration against human labels, LLM-based structured extraction) to keep building skills in exactly the area I am targeting.

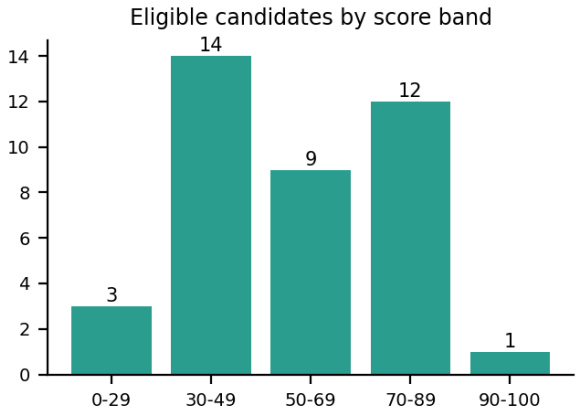
5. Results on the provided dataset

Scoring mode for this run: **deterministic (no LLM configured)**. 35 profiles enriched with live GitHub data.



Top 10 eligible candidates by score, broken down by category.





Why candidates were rejected

Rejection reason	Candidates
No AI project evidence	8
No Python evidence	7
AI terms only in skills list	1

A candidate can have more than one reason.

Top of the shortlist

#	Candidate	Score	Strongest AI work (from the generated summary)
1	Prathamesh Patil	91.5	Adaptive Agentic RAG - Multi-Agent Research System (deep: agents/tool-calling, backend integration, data processing, evaluation/metrics, retrieval/RAG)
2	Yash Maini	88.0	NetoAI Solutions (deep: agents/tool-calling, backend integration, data processing, evaluation/metrics, retrieval/RAG, state/memory, training/fine-tuning)
3	V Sree Raghu Vardhan	86.5	Darwinbox – Hyderabad, India (deep: agents/tool-calling, backend integration, data processing, evaluation/metrics, retrieval/RAG, state/memory)
4	Jyandeep Baishya	86.0	Agentic RAG Assistant (deep: agents/tool-calling, backend integration, evaluation/metrics, retrieval/RAG, state/memory)
5	Prajwal A S	84.0	Project Associate - I (deep: agents/tool-calling, backend integration, data processing, evaluation/metrics, retrieval/RAG, state/memory)

6. Responsible use and limitations

This is a **decision-support tool for the first pass**, not a hiring decision maker. A human should review the shortlist and any rejection before it affects a candidate.

Limitation	Mitigation
Section and project detection is heuristic; unusual layouts can merge two projects.	Merging is preferred over splitting so nobody is penalised for a parsing fragment; per-project verdicts are visible for review.
Keywords cannot verify that claims are true or high quality.	Optional LLM judgment with evidence, GitHub signals, and concerns that feed interview questions.
Scanned image-only PDFs have no text layer.	Reported as unreadable rather than silently scored as zero.
Weights reflect one role's priorities.	All weights and thresholds live in configuration; calibration against human labels is the first planned improvement.
Automated screening can encode bias.	No personal attributes (name, college, location) are used in scoring; only technical evidence is scored, and every decision is explainable.

7. Next steps

Improvement	Value
Calibrate weights against ~20 human-scored resumes	Measurable agreement with reviewers
LLM-based structured extraction of projects	More accurate project-level verdicts
Verify linked repos against resume claims	Stronger evidence of ownership
Reviewer UI with override feedback loop	Faster review and continuous improvement